

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testin

COURSE CODE: CSA3711

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Code of Conduct:

I P.Ranjithkumar (192411119)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:P.Ranjithkumar

1. A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.

Testing as an Engineering Activity

A banking application needs proper testing because random testing may miss important errors. Testing should be done in a planned way.

Testing Process

Requirement



Test Planning



Test Case



Test Execution



Find Defect



Fix and Retest

We can use boundary testing, equivalence testing and decision table testing. Tools like Jira can be used to record defects.

Example:

For money transfer, test ₹100, ₹1,000, ₹0, wrong account number and amount above the limit.

This type of testing finds more defects and improves the quality of the banking application.

2. An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.

Testing Process for ATM Withdrawal

ATM testing should be done step by step instead of testing randomly.

Requirement



Test Case



Execute



Report Defect



Retest



Final Testing

For ATM withdrawal, we can test:

Correct card and PIN

Wrong PIN

Insufficient balance

Invalid amount

Maximum withdrawal limit

Cash not available

Example:

If the account has ₹5,000 and the user withdraws ₹2,000, the ATM should give ₹2,000 and show the remaining balance as ₹3,000.

A proper testing process reduces mistakes and makes the ATM more reliable.

3. A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.

Error, Defect and Failure

These three terms are different.

Human Mistake



Error



Defect



Failure

Error: A developer makes a mistake while writing code.

Defect: The mistake remains in the software.

Failure: The user gets a wrong result when the software runs.

Example:

A developer writes wrong login logic. The code contains a defect. When a correct user cannot log in, it becomes a failure.

Defect reports should clearly mention the expected result and actual result. This helps developers understand the problem easily.

4. A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.

Defect Clustering

Defect clustering means that many defects are usually found in a few parts of the software.

Application

— Login	→ 2 defects
— Payment	→ 10 defects
— Search	→ 1 defect
— Checkout	→ 7 defects

So, testers should give more attention to the Payment and Checkout modules.

Example

In an online shopping website, payment may have more defects because it connects with banks and payment gateways.

By testing high-defect areas first, important defects can be found quickly and testing time can be saved.

5. A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.

Defect Repository

A defect repository is a place where all defects are stored.

Find Defect



Record Defect



Fix Defect



Retest



Store Result

Defects can be grouped as:

Functional

UI

Database

Performance

Security

Example:

If a payment error occurs again and again, the old defect records can be checked. The tester can create new test cases for the same problem.

This helps prevent the same defect in future releases.

6. A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.

Mobile Banking Fund Transfer

If a banking app crashes during fund transfer, the tester should check different situations.

Enter Account



Check Balance



Enter Amount



Transfer



Confirmation

Test cases should include:

Wrong account number

Low balance

Large amount

No internet

Server problem

Double clicking Transfer

Example:

If the user clicks Transfer twice, money should not be deducted twice.

Proper testing can find the cause of the crash and prevent similar problems later.

7. A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.

Testing and Debugging

Testing and debugging are not the same.

Testing

Finds the problem

Done mainly by tester

Reports the defect

Testing

↓

Defect Found

↓

Debugging

↓

Defect Fixed

↓

Retesting

Example:

If login crashes, the tester reports it. The developer checks the code and fixes the error.

Then the tester tests it again.

Both testing and debugging are needed for good software quality.

Debugging

Finds the cause

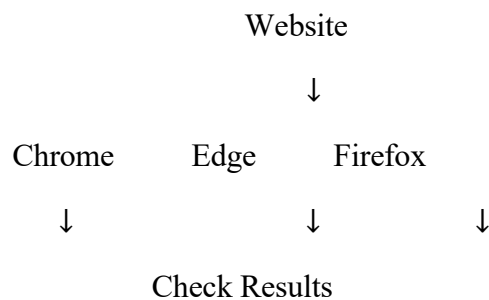
Done mainly by developer

Fixes the defect

8. An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.

Cross-Browser Testing

A website may work correctly in one browser but not in another browser.



We should test:

Login

Search

Cart

Payment

Buttons

Page design

Example:

The payment button may work in Chrome but not work in Firefox.

Tools like Selenium can help test the website on different browsers.

Cross-browser testing gives users a better experience.

9. A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.

Payroll Calculation Defects

If a payroll system calculates salary incorrectly, it is mainly a logic or functional defect.

Example:

Basic Salary = ₹20,000

Allowance = ₹5,000

Expected = ₹25,000

Actual = ₹20,000

The calculation is wrong.

Testers should check:

Salary

Allowance

Tax

Deduction

Bonus

Overtime

Boundary testing and proper calculation test cases can help prevent these defects.

10. A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.

Random Testing vs Planned Testing

Random testing may miss important cases. Test cases should be prepared before testing.

Requirement



Test Cases



Testing



Defect



Retesting

For example, if an age field accepts 18 to 60, test:

17 → Invalid

18 → Valid

30 → Valid

60 → Valid

61 → Invalid

This is better than entering random values.

Using boundary testing and equivalence testing improves defect finding.

11. A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.

Data Inconsistency in Healthcare Application

A healthcare application contains important patient data. The same data should be correct in all modules.

Patient Details



Database



Billing Medical Record

If the patient's name is changed in one module but not changed in another module, there is a data inconsistency.

Possible reasons are:

Database error

API error

Wrong field mapping

Update failure

Testing should check whether the same patient information is correctly updated everywhere.

Clear defect reports can help developers fix these problems.

12. A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.

Repeated Defects

If the same defects appear again and again, the defect repository should be checked.

Defects



Repository



Find Common Problems



Create New Test Cases



Prevent Repeated Defects

Example:

If login defects are found in every release, testers should create more login test cases.

They can test:

Wrong password

Empty fields

Locked account

Password expiry

Multiple wrong attempts

This reduces repeated defects in future releases.

13. A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.

Learning System Failure During Peak Usage

A website may work properly with a few users but fail when many users access it at the same time.

Few Users



Normal Testing



Many Users



Load Testing



Check Performance

For a learning website, test:

Many students logging in

Many students watching videos

Many students submitting exams

Many database requests

Example:

If the website works for 100 users but crashes with 5,000 users, there is a performance problem.

Load testing and stress testing can help find these problems.

14. A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.

Tester Involvement in SDLC

Testers should be involved from the beginning of the project.

Requirement



Design



Coding



Testing



Release

If testers join during the requirement stage, they can find missing or unclear requirements early.

Example:

Suppose a banking requirement does not mention the daily transfer limit. A tester can identify this before coding starts.

Early tester involvement saves time and reduces defects.

15. A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.

Incomplete Test Coverage

Sometimes software works for normal cases but fails for other cases because all situations were not tested.

For a login page, we should test:

Correct username + Correct password

Wrong username

Wrong password

Empty username

Empty password

Both fields empty

Locked account

Good test coverage includes:

Normal cases

Wrong cases

Boundary cases

Special cases

Using different test cases gives better coverage and increases software reliability.

16. A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.

Defect Reporting

A defect report should be simple and clear.

Example

Defect ID : D101

Module : Delivery Tracking

Steps:

1. Login
2. Enter tracking number
3. Click Search

Expected:

Delivery details should appear.

Actual:

Error message is displayed.

A good report should contain:

Defect ID

Module

Steps

Expected result

Actual result

Severity

Clear reports help developers understand and fix the problem faster.

17. A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.

Consistency in Testing

Different modules should follow the same testing process.

Common Test Plan



Common Test Cases



Common Testing Process



Common Defect Report

For example, all modules can use the same test case format:

Test ID	Input	Expected	Result
TC01	Valid data	Success	Pass
TC02	Wrong data	Error	Pass

Using common formats, checklists and reviews makes testing consistent across the project.

18. A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.

Edge Cases and Boundary Testing

Edge cases are unusual values or values near the limit.

Suppose an ATM allows withdrawal from ₹100 to ₹50,000.

₹99 → Invalid

₹100 → Valid

₹101 → Valid

₹49,999 → Valid

₹50,000 → Valid

₹50,001 → Invalid

These values should be included in testing.

Boundary Value Analysis helps find problems near minimum and maximum limits.

Therefore, test cases should include values below, at and above the boundary.

19. A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.

Defect Documentation

Without proper defect documentation, it becomes difficult to know which problems were already fixed.

Defect Found



Document



Assign



Fix



Retest



Close

A defect record should contain:

ID

Description

Module

Steps to reproduce

Expected result

Actual result

Severity

Status

Example:

If the same payment problem occurs again, the tester can check the old record and understand what happened previously.

Good documentation makes defect tracking easier.

20. A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.

Banking Transaction Testing

Banking transactions need careful testing because mistakes can cause money loss.

Transaction



Check Account



Check Balance



Check Amount



Process



Update Balance



Confirmation

Test cases should include:

Correct transaction

Wrong account

Insufficient balance

Amount above limit

Network failure

Duplicate transaction

Example:

If the network stops after money is transferred, the user should not be charged twice when trying again.

Using boundary testing, negative testing, integration testing and regression testing can reduce transaction failures.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps

